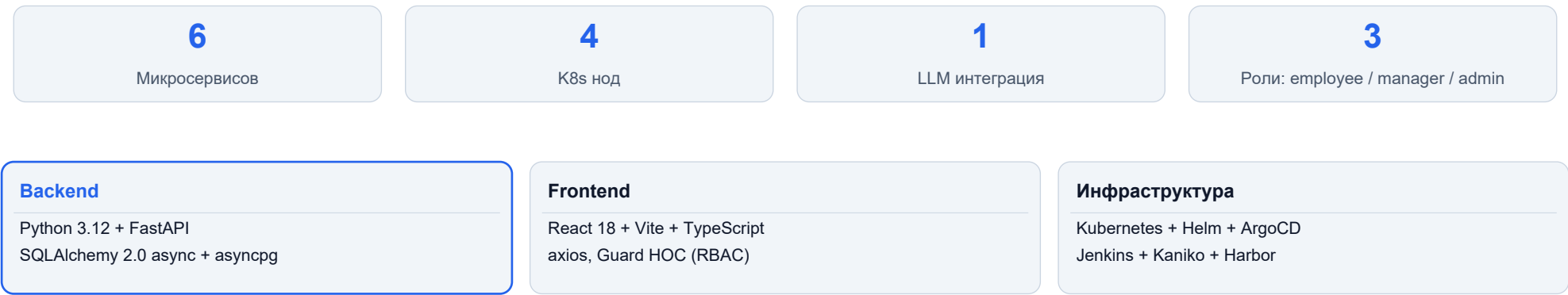


ДИПЛОМНАЯ РАБОТА

Проектирование, разработка и развёртывание в self-hosted Kubernetes



Проблема

Отсутствие автоматизированного инструмента тестирования сотрудников с LLM-анализом слабых мест и персональными рекомендациями.

Целевой сценарий

1. Сотрудник проходит тест через браузер
2. LLM-сервис анализирует ответы
3. Report-сервис генерирует PDF/HTML отчёт
4. Руководитель изучает рекомендации

Выполненные задачи

- Микросервисная архитектура (6 сервисов)
- REST API + JWT + Internal JWT
- LLM-интеграция GigaChat / mock
- PDF-отчёты с кириллицей
- Kubernetes + Helm деплой
- CI/CD: Jenkins + Kaniko + ArgoCD
- React SPA с RBAC
- NetworkPolicy: default-deny

Готово

Готово

Готово

Готово

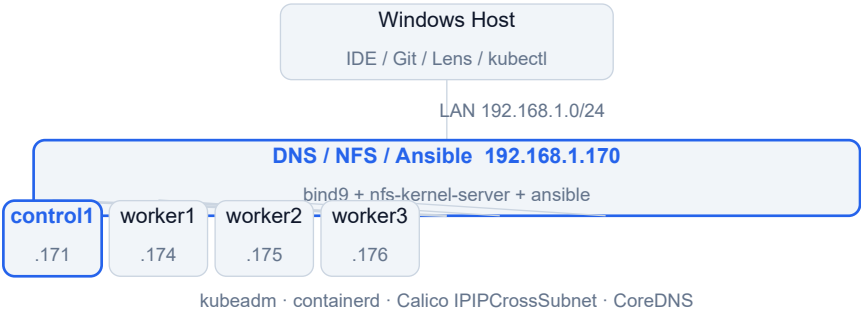
Готово

Готово

Готово

Готово

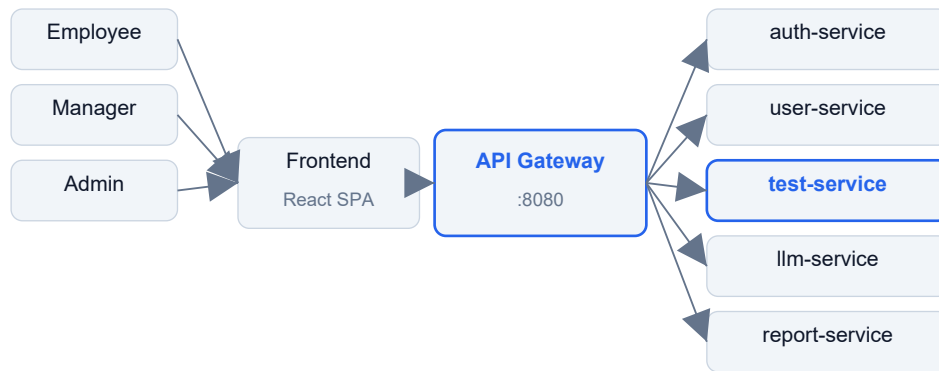
Топология кластера



Компоненты платформы

Компонент	Роль
kubeadm	Bootstrap кластера
containerd	Container runtime
Calico	CNI + NetworkPolicy
CoreDNS + NodeLocalDNS	Кластерный DNS
MetalLB	LoadBalancer IP
ingress-nginx	HTTP/HTTPS Ingress
cert-manager	TLS сертификаты
nfs-subdir	StorageClass nfs-client
Harbor	Container Registry
ArgoCD	GitOps CD
Jenkins	CI Pipeline

Контекстная диаграмма



Все сервисы → PostgreSQL (schema-per-service)

Монорепозиторий и принципы

pkg/

[services/](#)

frontend/

deploy/

- Один `pyproject.toml`, `uv` — менеджер зависимостей
- Per-service extras в `optional-dependencies`
- 12-factor: конфиг через `env`, логи в `stdout` JSON
- Stateless сервисы — данные только в PostgreSQL
- Non-root контейнеры, `readOnlyRootFilesystem`

Сервис	Порт	Схема БД	Ответственность
api-gateway	8080	—	JWT-валидация, проксирование, централизованный формат о
auth-service	8080	auth	Регистрация, логин, refresh-токены, bcrypt хэширование
user-service	8080	users	Профили, роли, отделы, должности сотрудников
test-service	8080	tests	Тесты, вопросы, назначения, попытки — оркестратор submi
llm-service	8080	llm	Анализ ответов: GigaChat (prod) / mock (dev)
report-service	8080	reports	Генерация PDF/HTML отчётов (xhtml2pdf + ReportLab)
frontend	8080	—	React SPA на nginx-unprivileged, VITE_API_URL → api-gat

Healthcheck endpoints

GET /healthz — liveness (процесс жив)
GET /readyz — readiness (БД доступна)

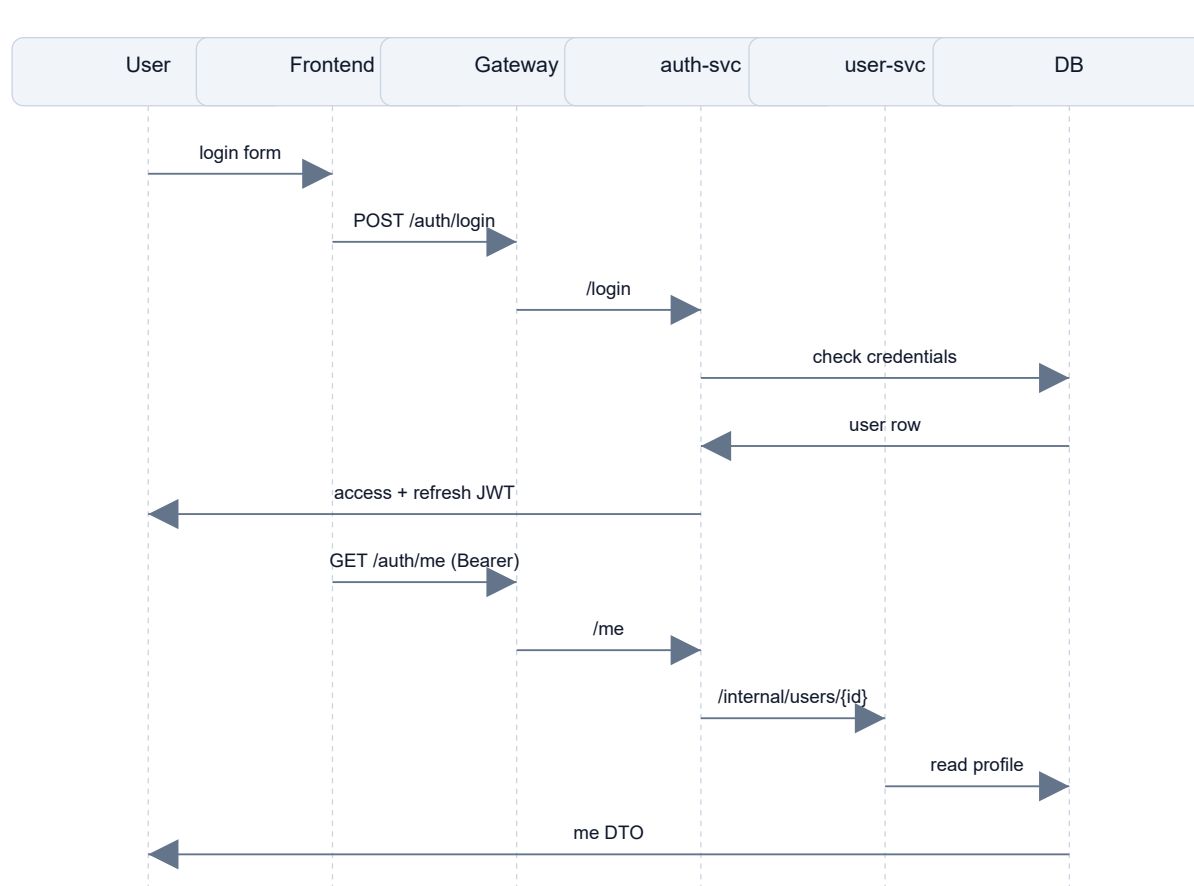
Internal JWT (service-to-service)

Header: X-Internal-Token
HS256, TTL 300 секунд

Миграции (Alembic)

Per-service миграции K8s Job (PreSync hook)

Слой	Технология	Роль	Слой	Технология	Роль
Runtime	Python 3.12	Язык бэкенда	Frontend	React 18 + Vite + TS	SPA
Web	FastAPI + Uvicorn	ASGI-фреймворк	HTTP	axios	API-клиент
ORM	SQLAlchemy 2.0 async	Async ORM	Lint	ruff	Linter + formatter
DB driver	asyncpg	Async PostgreSQL	Tests	pytest	Unit / integration
Migrations	Alembic	Версионирование схемы	PDF	xhtml2pdf + ReportLab	PDF генерация
Auth	PyJWT + passlib[bcrypt]	JWT + хэширование	Templates	Jinja2	HTML шаблоны
HTTP client	httpx	Async межсервисные вызовы	Deps	uv	Package manager
Config	pydantic-settings	12-factor env-конфиг	Build	Kaniko	OCI-сборка без Docker
Logging	structlog	JSON-логи в stdout	Registry	Harbor	Container registry
Metrics	prometheus-client	Prometheus /metrics	IaC	Helm + ArgoCD	K8s charts + GitOps



Детали реализации

Пользовательский JWT

access_token: HS256, TTL 15 мин
refresh_token: TTL 7 дней (в БД)

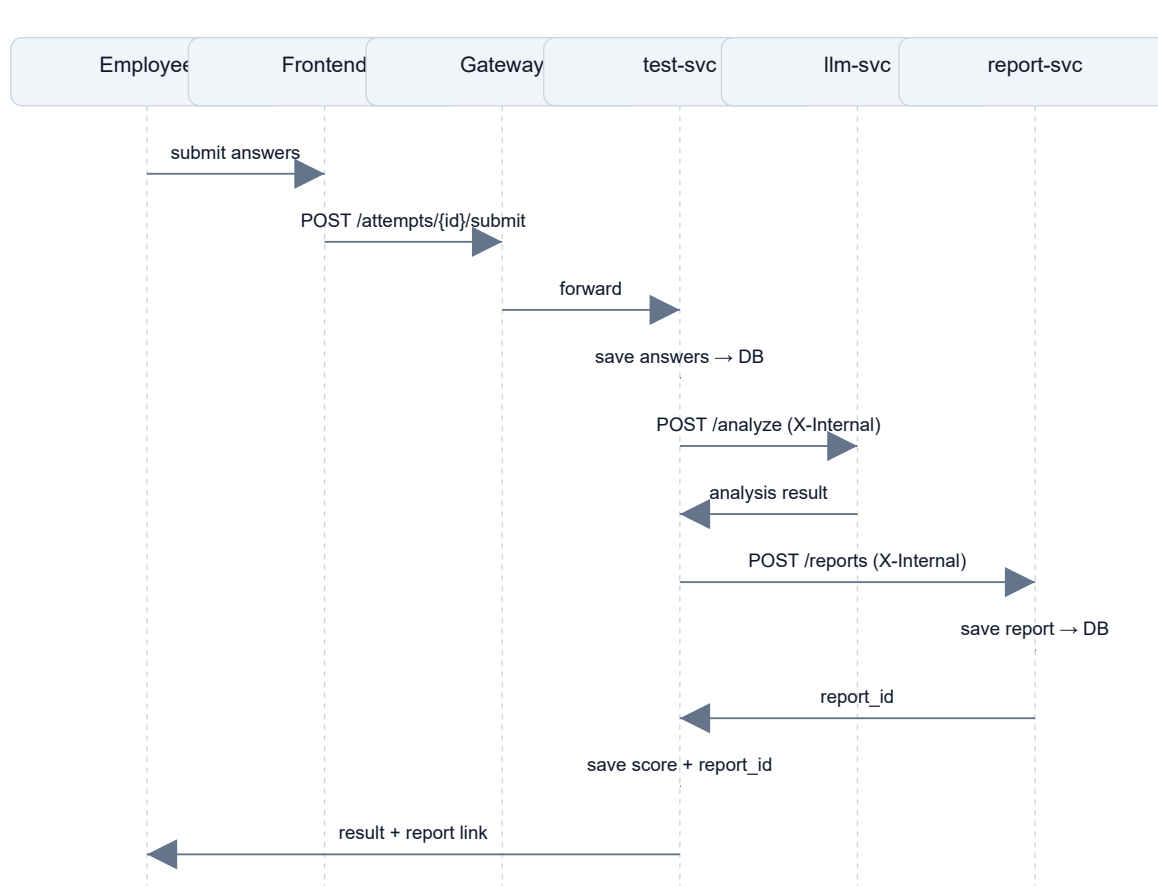
Internal JWT

Заголовок X-Internal-Token
TTL 300 с, отдельный секрет

Хранение паролей

bcrypt через passlib

Соль генерируется автоматически



test-service — оркестратор

Единственный знает о полном бизнес-процессе.
Ilm-service и report-service независимы

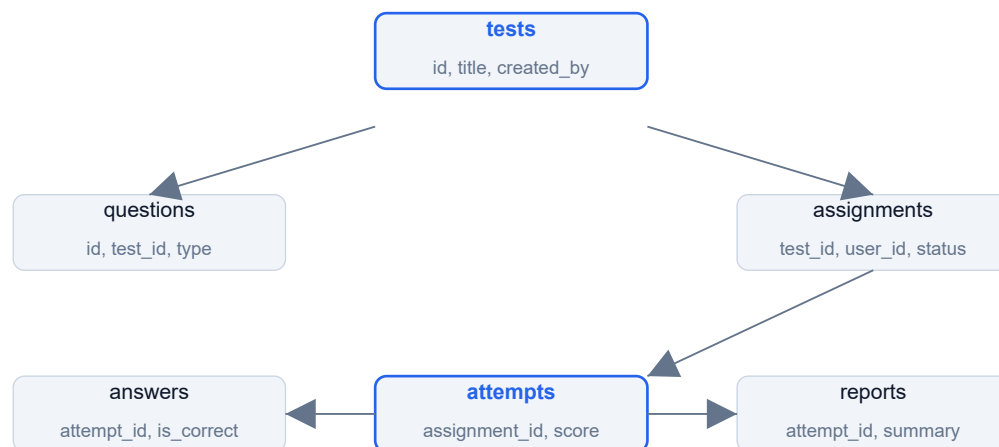
Статусы назначения

Статус	Описание
assigned	Тест назначен
in_progress	Сотрудник начал
completed	Завершён, отчёт готов

LLM-провайдеры

ENV	Поведение
mock	Детерминированный анализ (dev)
gigachat	Реальный GigaChat API (OAuth2)

Ключевые сущности (схема tests)



Все схемы PostgreSQL

Схема	Сервис	Ключевые таблицы
auth	auth-service	sessions
users	user-service	users, departments
tests	test-service	tests, questions, assignments, attempts, answers

Решение: Enum в PostgreSQL

SQLAlchemy по умолчанию сохраняет NAME (FREE_TEXT), PostgreSQL ожидает VALUE (free_text).

NetworkPolicy (default-deny)

Политика	Эффект
default-deny-all	Блокирует весь ingress/egress
allow-dns	UDP/TCP 53 → kube-system
allow-internal	Трафик внутри namespace app
allow-from-ingress	ingress-nginx → frontend + api-gw
allow-postgres-egress	app → database ns :5432
allow-llm-egress-external	llm-service → Internet :443/9443

Только llm-service может выходить в Интернет (GigaChat API)

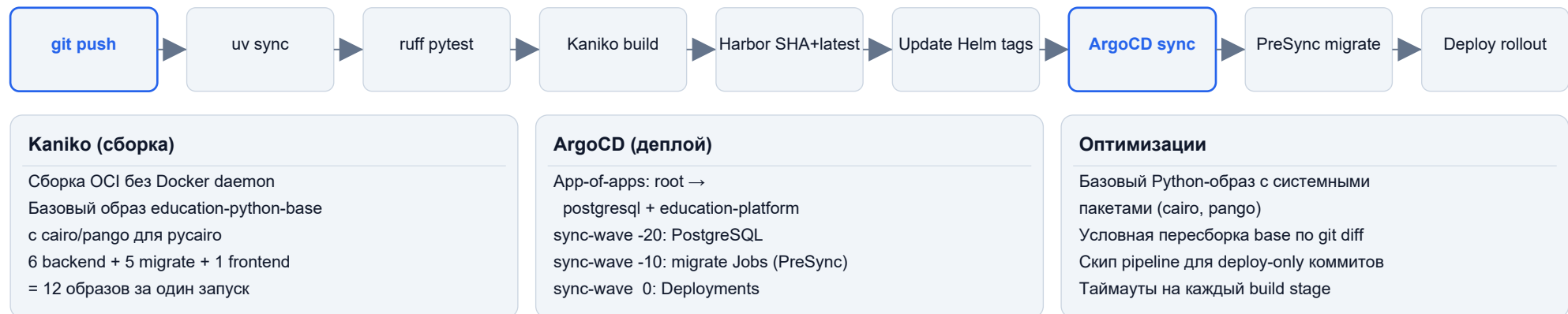
Все остальные сервисы заблокированы от внешних соединений.

Container Security Context

Параметр	Значение
runAsNonRoot	true
runAsUser / runAsGroup	10001 / 10001
readOnlyRootFilesystem	true
allowPrivilegeEscalation	false
capabilities.drop	ALL
seccompProfile	RuntimeDefault

Kubernetes Secrets

Secret	Содержит
jwt-secret	Ключ пользовательского JWT
internal-jwt-secret	Ключ service-to-service JWT
gigachat-creds	OAuth2 GigaChat
auth-db / users-db	Per-service DB credentials



Неймспейсы

Namespace	Содержит
app	6 backend сервисов + frontend
database	PostgreSQL StatefulSet
ingress-nginx	Ingress controller
argocd	ArgoCD
jenkins	Jenkins
harbor	Container Registry
cert-manager	TLS cert manager

Ingress (TLS)

Host	Service
app.mokryakov.local	frontend :8080
api.mokryakov.local	api-gateway :8080

Каждый Deployment

requests: CPU 50m / RAM 96Mi
limits: CPU 500m / RAM 512Mi
livenessProbe: GET /healthz
readinessProbe: GET /readyz
securityContext: non-root, readOnly FS
capabilities.drop: ALL
ServiceAccount: education-platform
imagePullSecrets: harbor-pull-secret

ArgoCD sync-waves

Wave	Ресурс
-20	PostgreSQL (отдельный Helm release)
-10	Migrate Jobs (PreSync hook)
0	Deployments + Services + Ingress

Enum mismatch в PostgreSQL

SQLAlchemy

Симптом: `InvalidTextRepresentationError: FREE_TEXT`
SQLAlchemy пишет NAME, PostgreSQL ожидает VALUE.
Решение: `values_callable=_enum_values +`
`validate_strings=True` во всех Enum-колонках.

MissingGreenlet (async lazy load)

SQLAlchemy

Симптом: `greenlet_spawn` has not been called
`len(t.questions)` вне `async`-сессии → `lazy load`.
Решение: заменён на `scalar_subquery()` —
`count` считается в основном SQL-запросе.

Кириллица в PDF отчётах

PDF/ReportLab

Симптом: все русские буквы → `nnnn...`
`xhtml2pdf` не применял CSS `@font-face`.
Решение: явная регистрация `DejaVuSans`:
`pdfmetrics.registerFont(TTFont(...))` перед `pisa`.

русcairo в Kaniko (apt системные пакеты)

Kaniko/Docker

Симптом: `pkg-config for cairo not found`
`русcairo` требует `cairo/pango` при сборке.
Решение: базовый образ `education-python-base`
с предустановленными системными пакетами.

Страницы по ролям (RBAC)

Роль	Страницы
employee	Список назначений, прохождение теста, результаты, профи
manager	Создание/редакт. тестов, назначение, просмотр попыток/o
admin	Управление пользователями + все страницы manager

Архитектурные решения

- Guard HOC — защита роутов по роли из JWT
- axios instance с Bearer token из localStorage
- Централизованный API-слой (api.ts)
- Reset setError(null) + setData(null) в useEffect при смене route (fix: страницы без refresh)

Сборка и деплой

npm run build → статика в nginx-unprivileged
API URL через VITE_API_URL (build arg)
Ingress: app.mokryakov.local → frontend pod

Исправленная проблема

Страницы не загружались без refresh

При SPA-навигации старые error-состояния блокировали новые fetch-запросы.
Решение: явный сброс setError(null) +

6

Backend микросервисов

12

Docker образов в Harbor

5

Независимых схем БД

100%

Критериев готовности

GitOps

Стратегия деплоя

0

Секретов в коде

Что реализовано

- Production-ready микросервисная архитектура на Python/FastAPI
- Полный CI/CD: Jenkins + Kaniko + Harbor + ArgoCD
- Self-hosted Kubernetes с GitOps (ArgoCD app-of-apps)
- Security-by-default: NetworkPolicy, non-root, drop ALL caps
- LLM-интеграция с подменяемым провайдером (mock / GigaChat)
- Prometheus-метрики, JSON-логи, healthcheck на каждом сервисе

Потенциал развития

- Асинхронная очередь (NATS/Kafka) вместо синхронного HTTP
- Мониторинг: Prometheus + Grafana + Loki + Alertmanager
- Horizontal Pod Autoscaler по RPS
- Разделение PostgreSQL по физическим экземплярам
- Rate limiting и circuit breaker на API Gateway
- RBAC на уровне Kubernetes

Вывод

Разработана и задеплоена полноценная производственная платформа с микросервисной архитектурой,